



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Практическая работа № 8/2 ч.

Разработка мобильных приложений

	(наименование дисциплины (модуля) в соответствии с учебным планом)
Уровень	бакалавриат
	(бакалавриат, магистратура, специалитет)
Форма обучения	очная
	(очная, очно-заочная, заочная)
Направление(-я) подготовки	09.03.02 «Информационные системы и технологии»
	(код(-ы) и наименование(-я))
Институт	кибербезопасности и цифровых технологий
	(полное и краткое наименование)
Кафедра	КБ-14 «Цифровые технологии обработки данных»
	(полное и краткое наименование кафедры, реализующей дисциплину (модуль))
Используются в данной редакции с учебного года	2023/24
	(учебный год цифрами)
Проверено и согласовано « ____ » _____ 20__ г.	
	(подпись директора Института/Филиала с расшифровкой)

Москва 2024 г.

ОГЛАВЛЕНИЕ

1	КАРТОГРАФИЧЕСКИЕ СЕРВИСЫ.....	3
1.1	Google Maps.....	4
1.2	Яндекс.Карты	5
1.3	OpenStreetMap	6
1.4	Here	7
1.5	2ГИС	7
2	ИНТЕГРАЦИЯ YANDEXMAPS В ПРИЛОЖЕНИЕ	9
2.1	Определение местоположения пользователя.....	12
2.2	Построение маршрута движения.	15
3	ИНТЕГРАЦИЯ OPENSTREETMAP В ПРИЛОЖЕНИЕ	19
4	КОНТРОЛЬНОЕ ЗАДАНИЕ	24

1 КАРТОГРАФИЧЕСКИЕ СЕРВИСЫ

Современное развитие картографических сервисов направлено на повышение точности карт, скорости построения маршрутов, интеграции функций дополненной реальности и т.д. Наиболее популярным сервисом, используемым в приложениях для потребителей из России и стран СНГ, являются карты от компании Яндекс. При ориентации приложений на западные страны используются международные сервисы типа «*Google maps*». Чтобы определиться с поставщиком карт, требуется учитывать множество различных параметров. Некоторые сервисы не поддерживают работу в условиях отсутствия интернета, построение маршрутов движения, а также нанесение пользовательских данных на карту. В некоторых случаях требуется, чтобы карты были растровыми, а где-то векторными для высококачественного масштабирования.

К наиболее распространенным картографическим сервисам относятся:

- «*Google Maps*» – картографический сервис корпорации «*Google*». Во многих городах и странах является самыми точными и детализированными сервисом. Сервис представляет собой карту и спутниковые снимки планеты Земля. Для многих регионов доступны высококачественные аэрофотоснимки (выполненные с высоты 250 – 500 м), а также просмотра под углом 45° с четырёх сторон света. Доступны 20 уровней масштаба.

- «*Apple Maps*» – сервис от компании «*Apple*» для операционных систем «*iOS*» и «*macOS*». Имеются возможности просмотра картографических схем и спутниковых снимков, построения пошаговой навигации с помощью ассистента «*Siri*» с голосовым сопровождением, поддержка 3D-режима.

- Яндекс Карты – сервис от компании Яндекс, поддерживающий функционал поиска по карте, отображения информации о пробках, отслеживания городского транспорта, построения маршрутов, отображения панорам улиц городов.

- «*HERE*» – картографический сервис компании «*HERE Technologies*». Поддерживаются функции поиска по карте, отображения спутниковых снимков и 3D-карт, построения маршрутов, отражения пробок.

- «*OpenStreetMap*» – некоммерческий картографический сервис, созданный силами сообщества пользователей. Наполнение карт осуществляется данными с пользовательских GPS-трекеров, аэрофотографий, видеозаписей, спутниковых снимков и панорам улиц, а также знаний пользователя, рисующего карту. Любой зарегистрированный пользователь имеет права вносить изменения в карту.

- «*Azure Maps*» – набор служб геопространственной картографии. Поддержка векторных и растровых карт, а также спутниковых изображений, геозоны, построение маршрута и т.д.

- «*ArcGIS*» – набор геоинформационных программных продуктов компании «*ESRI*». Продукты «*ArcGIS*» позволяют разрабатывать собственные приложения картографирования и пространственного анализа. Сервис позволяет создавать карты, поддерживает построение маршрутов, отображение пробок и т.д.

В случаях, когда разрабатываемое приложение свободно распространяется – карты предоставляются условно бесплатно. В случаях разработки программ для внутреннего использования или для коммерческого распространения (например, платное приложение для финансистов) требуется использование коммерческой лицензии. Стоит уделять особое внимание, какую долю в функциональности занимает картографический сервис и какое прогнозируемое количество обращений к нему со стороны пользователей в месяц.

1.1 Google Maps

Сервис Google Maps является лидером среди современных картографических сервисов в области спутниковых и аэрофотоснимков, а также по количеству различных дополнительных сервисов и инструментов («*Google Earth*», «*Google Mars*», разнообразные погодные и транспортные сервисы, одно из самых мощных API). В «*Google Maps Android API*» имеются следующие возможности:

- поддержка фрагментов;
- расширенные слои карт, включая спутниковый, гибридный, пробки, внутренний слой для основных аэропортов и торговых центров;
- уменьшение строк кода, для создания маркеров и информационных окон;

- настройка компонентов интерфейса и жестов взаимодействия с картой;
- добавление многоугольников, ломаных линий и кругов на карту;
- добавление изображений, привязанных к определенной широте и долготе, которые будут перемещаться при перетаскивании и масштабировании карты;
- поддержка носимых устройств с помощью.

Выбор требуемого API для решения конкретной задачи возможно осуществить с помощью информационного сервиса по адресу: [«https://developers.google.com/maps/documentation/api-picker»](https://developers.google.com/maps/documentation/api-picker).

В 2018 г. разработчикам был представлен «*Google Maps Platform*» сервис, объединивший 18 инструментов в три категории: «Карты», «Маршруты» и «Места». Использование платформы возможно условно-бесплатное (в пределах \$200) и платное. Стоимость размещена на сайте: [«https://mapsplatform.google.com/pricing»](https://mapsplatform.google.com/pricing).

Стоит отметить, что для доступа к услугам «*Google Maps Platform*» разработчикам требуется действующий ключ API и платежный аккаунт на «*Google Cloud Platform*».

1.2 Яндекс.Карты

Является наиболее распространённым картографическим сервисом на территории России и стран СНГ. Сервис позволяет интегрировать карты в мобильные приложения для ОС «*Android*» и «*iOS*» с помощью инструментария «*Yandex MapKit*». Возможно использование карт как с подключением к интернету, так и в его отсутствии.

«*Yandex MapKit*» позволяет интегрировать в разрабатываемое приложение практически любую функцию приложения «Яндекс.Карты»: поиск адресов, прокладка маршрута с учетом строящихся домов, отображение пробок, название организаций и т.д. Сервис предоставляется бесплатно и на коммерческой основе в зависимости от используемого функционала. Платная лицензия выдаётся на год, а ее стоимость зависит от назначения приложения, его возможностей и количества запросов к сервису за сутки. Ограничения и стоимость указана в информационном ресурсе по адресу:

«https://yandex.ru/dev/maps/mapkit/doc/intro/concepts/enterprise.html#enterprise_limits».

Использование «*MapKit*» возможно бесплатно при выполнении следующих условий.

- используется в бесплатных приложениях, которые может скачать любой человек (запрет на закрытые и платные приложения);
- запрещается использовать для ведения по маршруту и в проектах по мониторингу и диспетчеризации;
- выполняется работа с картой только в онлайн-режиме;
- отсутствует функционал мониторинга транспорта;
- все данные, полученные через «*MapKit*», должны отображаться на карте.

Сохранять или изменять информацию нельзя;

- общее число запросов к сервисам поиска, маршрутизации и просмотра панорам в сутки не должно превышать 25 тысяч.

Помимо основной библиотеки «*MapKit*» доступны дополнительные библиотеки, расширяющие возможности вашего приложения:

- «*MapKit Directions*» – используется для построения маршрута движения автомобиля;
- «*MapKit Places*» – используется для работы с панорамами;
- «*MapKit Search*» – используется для поиска объектов на карте и для функций геокодирования;
- «*MapKit Transport*» – используется для построения маршрутов для пешеходов, велосипедов, а также маршрутов с использованием общественного транспорта.

Библиотека «*MapKit*» доступна в репозиториях «*Maven Central*» и «*Gradle*».

1.3 OpenStreetMap

Картографический сервис с открытым исходным кодом. Отображаемые данные создаются и вносятся пользователями сообщества. Основной целью сообщества пользователей «*OSM*» является не столько формирование карты, сколько создание базы данных на ее основе и наполнение объектов информацией. Поэтому на базе «*OSM*» возможно создание множества различных слоев для карт:

велосипедные маршруты, карты курортов с достопримечательностями и так далее.

Интеграция данных карт является бесплатной, но имеются определенные ограничения технического характера (например, ограничение количества запросов выполнения функций геокодинга в секунду к серверу «OSM»). Репозиторий: <https://github.com/osmdroid/osmdroid>

1.4 Here

Геолокационная платформа, основанная «Nokia» (на текущий момент принадлежит консорциуму, в который входят компании «Audi AG», «BMW Group», «Daimler AG», «Intel», «Navinfo», «NVIDIA», «Pioneer», «Bosh», «Continental»). Имеется возможность прокладки маршрута, отображения номера домов, информации о POI (point of interest, достопримечательности, магазины, офисы, заправки и так далее). В случаях, если приложение ориентировано на пешеходов или поиск организаций (особенно в небольших городах), Here в России не используется из-за неактуальной информации о POI. Для водителей и логистических сервисов «Here» является хорошим решением. Карты «Here» установлены в автомобилях от «Audi», «BMW», «Daimler». Автопроизводители, входящие в концерн, используют «Here» в качестве базы данных в системах автопилотирования, что вылилось в проект «HD Live Map». Отдельно стоит отметить, что имеется возможность бесплатного использования функций в рамках определенного количества запросов в месяц («<https://www.here.com/get-started/pricing#servicerates>»).

Репозиторий с примерами проектов: <https://github.com/heremaps/here-android-sdk-examples>

1.5 2ГИС

Отечественный картографический сервис с высокой степенью детализации слоев по России и не только. Стоит отметить наличие сведений из справочников 2ГИС по компаниям, поэтому платформа пригодится для приложений, ориентированных на пешеходов или отображения информации с каталогами фирм. Однако следует заранее проверять, актуальна ли в нужном городе справочная информация, все ли POI нанесены на карту.

Сервис является условно-бесплатным, т.е. использование для свободно скачиваемых открытых приложений. 2ГИС сотрудничает с компанией «Apple»: в приложение Карты на iOS были встроены данные об организациях в 290 городах, фотографии заведений и отзывы на них.

2ГИС

2 ИНТЕГРАЦИЯ YANDEXMAPS В ПРИЛОЖЕНИЕ

Требуется создать новый проект «ru.mirea.«фамилия».Lesson8». В меню «*File | New Project | Empty Views Activity*». Название модуля «*YandexMaps*»

Библиотека «*MapKit*» предназначена для предоставления интерфейса взаимодействия с картографическим сервисом от компании Яндекс и ориентирован для платформ «*Android*» версии 5.0 и выше. Для использования набора инструментов «*MapKit*» требуется:

1. Получить ключ для работы с «*MapKit*»:

- перейти в кабинет разработчика по адресу «*https://developer.tech.yandex.ru/*»;
- авторизоваться с учетной записью Яндекса или зарегистрировать новый аккаунт;
- выбрать «подключить API» и выбрать пакет «*MapKit-мобильный SDK*»;
- заполнить информацию о проекте, указать нужный тариф.

Дальнейшие инструкции придут на почту. После предоставления доступа необходимо создать «Новый ключ» и установить идентификатор проекта. Полученный ключ возможно использовать для работы с «*MapKit*» в нескольких приложениях.

2. Установить библиотеку «*MapKit*»:

- открыть файл «*settings.gradle*» проекта (рисунок 2.1) и проверить наличие в секции «*repositories*» «*Maven Central*».

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}
```

Рисунок 2.1 – Структура проекта

- открыть файл «*build.gradle*» модуля «*YandexMaps*» и добавить

зависимость:

```
dependencies {  
    implementation 'com.yandex.android:maps.mobile:4.3.1-full'  
}
```

– синхронизировать проект («*Sync Now*»). В случае, если синхронизация завершилась успешно, при компиляции библиотека будет добавлена в проект автоматически.

3. Настроить библиотеку «*MapKit*»:

– добавить карту в файл `activity_main`:

```
<?xml version="1.0" encoding="utf-8"?>  
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">  
    <com.yandex.mapkit.mapview.MapView  
        android:id="@+id/mapview"  
        android:layout_width="match_parent"  
        android:layout_height="match_parent"/>  
</androidx.constraintlayout.widget.ConstraintLayout>
```

– создать класс-наследник «*Application*» и установить API-ключ в методе «*onCreate*»:

```
public class App extends Application {  
    private final String MAPKIT_API_KEY = "ВАШ API-КЛЮЧ";  
  
    @Override  
    public void onCreate() {  
        super.onCreate();  
        // Set the api key before calling initialize on MapKitFactory.  
        MapKitFactory.setApiKey(MAPKIT_API_KEY);  
    }  
}
```

– добавить запись в файл «*AndroidManifest.xml*» о созданном классе и разрешение на доступ к интернету.

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        ...
        android:name=".App"
        android:theme="@style/Theme.Lesson8">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>

```

– инициализировать карту в методе «*onCreate*» и установить местоположение камеры на карте:

```

public class MainActivity extends AppCompatActivity {
    private ActivityMainBinding binding;
    private MapView mapView;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        MapKitFactory.initialize(this);
        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());
        mapView = binding.mapview;
        mapView.getMap().move(
            new CameraPosition(new Point(55.751574, 37.573856), 11.0f, 0.0f,
0.0f),
            new Animation(Animation.Type.SMOOTH, 0),
            null);
    }
}

```

– передать события «*onStart*» и «*onStop*» в «*MapKitFactory*» и «*mapView*». В противном случае «*MapKit*» не отобразит карту и не освободит ресурсы после того, как карта становится невидимым для пользователя.

```

@Override
protected void onStop() {
    super.onStop();
    // Вызов onStop нужно передавать экземплярам MapView и MapKit.
    mapView.onStop();
    MapKitFactory.getInstance().onStop();
}

@Override
protected void onStart() {
    // Вызов onStart нужно передавать экземплярам MapView и MapKit.
    super.onStart();
    mapView.onStart();
    MapKitFactory.getInstance().onStart();
}

```

4. Собрать и запустить приложение. Стоит отметить, что в случае отсутствия отображения карты, требуется дождаться активации API-ключа.

На рисунке 2.2 представлен экран приложения «*YandexMaps*».

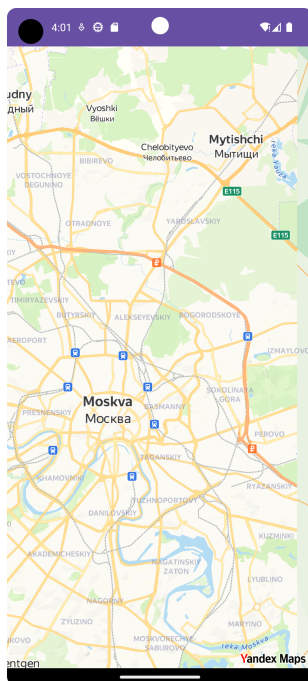


Рисунок 2.2 – Экран приложения «*YandexMaps*»

2.1 Определение местоположения пользователя

Основным предназначением картографических сервисов является определение местоположения пользователя и прокладка маршрута движения. Для организации данных возможностей требуется выполнить следующие шаги:

1. Указать требуемые разрешения для получения доступа к информации о местоположении устройства в «*AndroidManifest*»-файле:

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
<uses-permission android:name="android.permission.INTERNET"/>
```

Стоит отметить, что в случаях, когда приложению требуется получение данных о местоположении пользователя в фоновом режиме, необходимо добавить запрос на разрешение «*ACCESS_BACKGROUND_LOCATION*» (с версии «*Android Q*»). Доступ к данным о точном местоположении защищены разрешением – «*ACCESS_FINE_LOCATION*», средней точности – «*ACCESS_COARSE_LOCATION*».

По аналогии с практической работой № 5 модуль «*Camera*» произвести запрос на получение разрешений определения местоположения.

2. Настроить карту:

– реализовать интерфейс «*UserLocationObjectListener*», предназначенный для получения информации о новом местоположении пользователя;

```
public class MainActivity extends AppCompatActivity implements
    UserLocationObjectListener{
...
    @Override
    public void onObjectAdded(@NonNull UserLocationView userLocationView) {
    }

    @Override
    public void onObjectRemoved(@NonNull UserLocationView userLocationView) {
    }

    @Override
    public void onObjectUpdated(@NonNull UserLocationView userLocationView,
                               @NonNull ObjectEvent objectEvent) {
    }
}
```

– создать отдельный метод для настройки карты и произвести его вызов после проверки разрешений и получении успешного результата:

```
private void loadUserLocationLayer() {
    MapKit mapKit = MapKitFactory.getInstance();
    mapKit.resetLocationManagerToDefault();
    userLocationLayer = mapKit.createUserLocationLayer(mapView.getMapWindow());
    userLocationLayer.setVisible(true);
    userLocationLayer.setHeadingEnabled(true);
    userLocationLayer.setObjectListener(this);
}
```

– реализовать метод «*onObjectAdded*». В данном методе устанавливаются иконки устройства при определении местоположения и после того, как координаты были определены. В последней строчке кода формируется окружность, позволяющая пользователю определить точность координат.

```

@Override
public void onObjectAdded(UserLocationView userLocationView) {
    userLocationLayer.setAnchor(
        new PointF((float) (mapView.getWidth() * 0.5),
            (float) (mapView.getHeight() * 0.5)),
        new PointF((float) (mapView.getWidth() * 0.5),
            (float) (mapView.getHeight() * 0.83)));

    // При определении направления движения устанавливается следующая иконка
    userLocationView.getArrow().setIcon(ImageProvider.fromResource(
        this, android.R.drawable.arrow_up_float));

    // При получении координат местоположения устанавливается следующая иконка
    CompositeIcon pinIcon = userLocationView.getPin().useCompositeIcon();
    pinIcon.setIcon(
        "pin",
        ImageProvider.fromResource(this, R.drawable.search_result),
        new IconStyle().setAnchor(new PointF(0.5f, 0.5f))
            .setRotationType(RotationType.ROTATE)
            .setZIndex(1f)
            .setScale(0.5f)
    );

    userLocationView.getAccuracyCircle().setFillColor(Color.BLUE & 0x99ffffff);
}

```

3. Произвести сборку и запустить приложение. После запроса на разрешение доступа к информации о местоположении должен отобразиться рисунок 2.3.

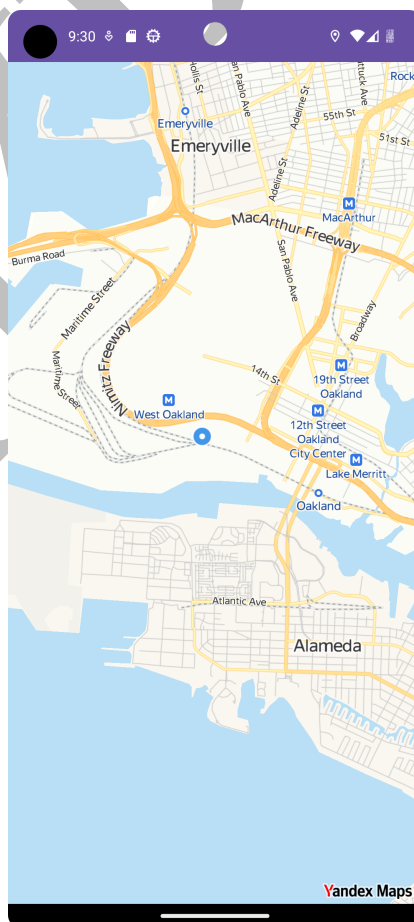


Рисунок 2.3 Экран приложения «YandexMaps»

2.2 Построение маршрута движения.

Создать новый модуль. В меню «*File | New | New Module | Phone & Tablet Module | Empty Views Activity*». Название модуля «*YandexDriver*».

В приложении указываются 2 точки и строится несколько маршрутов движения между заданными точками.

Для построения маршрута движения на карте требуется использовать класс «*DrivingRouter*» из пакета «*MapKit*». Основные этапы создания маршрутов:

1. Выполнение настроек библиотеки «*MapKit*» согласно разделу № 2.
2. Создание класса «*App*» и установка ключа для карты.
3. Добавление карты в файл разметки.
4. Реализация методов интерфейса «*DrivingSession.DrivingRouteListener*»

в классе «*MainActivity*».

```
public class MainActivity extends AppCompatActivity implements
DrivingSession.DrivingRouteListener {
    ...
    @Override
    public void onDrivingRoutes(@NonNull List<DrivingRoute> list) {
    }

    @Override
    public void onDrivingRoutesError(@NonNull Error error) {
    }
}
```

5. Инициализация карты. В поле класса устанавливаются значения координат начальной и конечной точек, массив, хранящий цвета для различных маршрутов.

```

public class MainActivity extends AppCompatActivity implements DrivingSession.Driv-
ingRouteListener {
    private ActivityMainBinding binding;
    private final String MAPKIT_API_KEY = "****";
    private final Point ROUTE_START_LOCATION = new Point(55.670005, 37.479894);
    private final Point ROUTE_END_LOCATION = new Point(55.794229, 37.700772);
    private final Point SCREEN_CENTER = new Point(
        (ROUTE_START_LOCATION.getLatitude() + ROUTE_END_LOCATION.getLatitude()) / 2,
        (ROUTE_START_LOCATION.getLongitude() + ROUTE_END_LOCATION.getLongitude()) /
2);
    private MapView mapView;
    private MapObjectCollection mapObjects;
    private DrivingRouter drivingRouter;
    private DrivingSession drivingSession;
    private int[] colors = {0xFFFF0000, 0xFF00FF00, 0x00FFBBBB, 0xFF0000FF};

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        MapKitFactory.initialize(this);
        DirectionsFactory.initialize(this);
        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());
        mapView = binding.mapview;
        mapView.getMap().setRotateGesturesEnabled(false);
        // Устанавливаем начальную точку и масштаб
        mapView.getMap().move(new CameraPosition(
            SCREEN_CENTER, 10, 0, 0));
        // Инициализируем объект для создания маршрута водителя
        drivingRouter = DirectionsFactory.getInstance().createDrivingRouter();
        mapObjects = mapView.getMap().getMapObjects().addCollection();
        submitRequest();
    }

    @Override
    protected void onStop() {
        mapView.onStop();
        MapKitFactory.getInstance().onStop();
        super.onStop();
    }

    @Override
    protected void onStart() {
        super.onStart();
        MapKitFactory.getInstance().onStart();
        mapView.onStart();
    }
}

```

6. Установка критериев к маршруту. В данном методе указывается количество альтернативных маршрутов, исходная точка и пункт назначения.

```
private void submitRequest() {
    DrivingOptions drivingOptions = new DrivingOptions();
    VehicleOptions vehicleOptions = new VehicleOptions();
    // Кол-во альтернативных путей
    drivingOptions.setRoutesCount(4);

    ArrayList<RequestPoint> requestPoints = new ArrayList<>();
    // Установка точек маршрута
    requestPoints.add(new RequestPoint(ROUTE_START_LOCATION,
        RequestPointType.WAYPOINT,
        null));
    requestPoints.add(new RequestPoint(ROUTE_END_LOCATION,
        RequestPointType.WAYPOINT,
        null));
    // Отправка запроса на сервер
    drivingSession = drivingRouter.requestRoutes(requestPoints, drivingOptions,
        vehicleOptions, this);
}
```

7. Обработка полученной информации от сервера. Метод интерфейса «onDrivingRoutes» возвращает список предложенных маршрутов, а метод «onDrivingRoutesError» – ошибки в ходе выполнения запроса построения маршрутов.

```
@Override
public void onDrivingRoutes(@NonNull List<DrivingRoute> list) {
    int color;
    for (int i = 0; i < list.size(); i++) {
        // настраиваем цвета для каждого маршрута
        color = colors[i];
        // добавляем маршрут на карту
        mapObjects.addPolyline(list.get(i).getGeometry().setStrokeColor(color);
    }
}

@Override
public void onDrivingRoutesError(@NonNull Error error) {
    String errorMessage = getString(R.string.unknown_error_message);
    if (error instanceof RemoteError) {
        errorMessage = getString(R.string.remote_error_message);
    } else if (error instanceof NetworkError) {
        errorMessage = getString(R.string.network_error_message);
    }
    Toast.makeText(this, errorMessage, Toast.LENGTH_SHORT).show();
}
```

Финальным этапом является процесс сборки и запуск приложения. На рисунке 2.3 представлен экран приложения «YandexDriver»

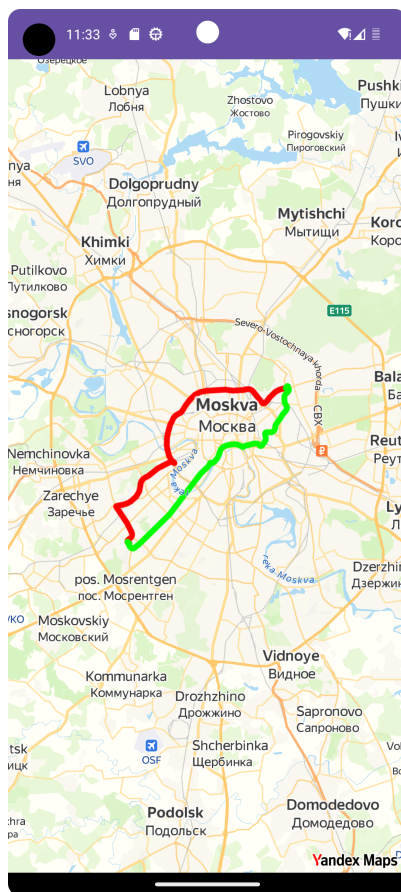


Рисунок 2.4 Экран приложения «YandexDriver»

Для **ВЫПОЛНЕНИЯ ЗАДАНИЯ** требуется построить маршрут от местоположения пользователя (в эмуляторе требуется в настройках указать координаты устройства) до любимого заведения. В конечной точке требуется установить маркет по нажатию на который происходит отображение краткой информации о заведении. Для создания и обработки нажатия на экран возможно использовать пример:

```
PlacemarkMapObject marker = mapView.getMap().getMapObjects().addPlacemark(new
Point(55.751574, 37.573856), ImageProvider.fromResource(this, R.drawable.icon));

marker.addTapListener(new MapObjectTapListener() {
    @Override
    public boolean onMapObjectTap(@NonNull MapObject mapObject, @NonNull Point
point) {
        Toast.makeText(getApplication(), "Marker click",
Toast.LENGTH_SHORT).show();
        return false;
    }
});
```

3 ИНТЕГРАЦИЯ OPENSTREETMAP В ПРИЛОЖЕНИЕ

Создать новый модуль. В меню «File | New | New Module | Phone & Tablet Module | Empty Views Activity». Название модуля «OSMMaps».

В приложении указываются 2 точки и строится несколько маршрутов движения между заданными точками.

В настоящее время, наиболее используемой альтернативой крупным картографическим коммерческим сервисам, является «OpenStreetMap». Для интеграции картографического сервиса требуется выполнить следующие шаги:

1. В большинстве случаев потребуется установка разрешений в AndroidManifest.xml:

```
<!-- скачивание карт -->
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<!-- определение местонахождения устройства -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
<!-- хранение скаченных карт -->
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="32" />
```

2. Установка библиотеки «osmdroid»:

– открыть файл «settings.gradle» проекта (рисунок 3.1) и проверить наличие в секции «repositories» «Maven Central».

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}
```

Рисунок 3.1 – Структура проекта

– открыть файл «build.gradle» модуля «OSMMaps» и добавить зависимость:

```
dependencies {
    implementation 'org.osmdroid:osmdroid-android:6.1.16'
    // библиотека для хранения данных SharedPreferences
    implementation "androidx.preference:preference:1.2.0"
}
```

– синхронизировать проект («Sync Now»). В случае, если синхронизация завершилась успешно, при компиляции библиотека будет добавлена в проект автоматически.

3. Настроить отображение карты:

– добавить карту в файл `activity_main`:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <org.osmdroid.views.MapView android:id="@+id/mapView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

– инициализировать карту и с помощью класса «*Configuration*» настроить карту:

```
public class MainActivity extends AppCompatActivity {
    private MapView mapView = null;
    private ActivityMainBinding binding;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Configuration.getInstance().load(getApplicationContext(),
        PreferenceManager.getDefaultSharedPreferences(getApplicationContext()));
        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());
        mapView = binding.mapView;
    }
    @Override
    public void onResume() {
        super.onResume();
        Configuration.getInstance().load(getApplicationContext(),
        PreferenceManager.getDefaultSharedPreferences(getApplicationContext()));
        if (mapView != null) {
            mapView.onResume();
        }
    }
    @Override
    public void onPause() {
        super.onPause();
        Configuration.getInstance().save(getApplicationContext(),
        PreferenceManager.getDefaultSharedPreferences(getApplicationContext()));
        if (mapView != null) {
            mapView.onPause();
        }
    }
}
```

После сборки проекта и запуска должен отобразиться типовой экран, представленный на рисунке 3.2.



Рисунок 3.2 Экран приложения «OSMMaps»

4. Для добавления кнопок масштабирования и возможности увеличения двумя пальцами используются следующие методы:

```
mapView.setZoomRounding(true);  
mapView.setMultiTouchControls(true);
```

5. Перемещение камеры в определенную точку требует доступа к контроллеру карты:

```
IMapController mapController = mapView.getController();  
mapController.setZoom(15.0);  
GeoPoint startPoint = new GeoPoint(55.794229, 37.700772);  
mapController.setCenter(startPoint);
```

6. Определение местоположения устройства:

- по аналогии с практической работой № 5 модуль «Camera» произвести запрос на получение разрешений определения местоположения;
- добавить слои «MyLocationNewOverlay» на карту:

```
locationNewOverlay = new MyLocationNewOverlay(new  
GpsMyLocationProvider(getApplicationContext()), mapView);  
locationNewOverlay.enableMyLocation();  
mapView.getOverlays().add(this.locationNewOverlay);
```

7. Добавление компаса:

```
CompassOverlay compassOverlay = new CompassOverlay(getApplicationContext(), new
InternalCompassOrientationProvider(getApplicationContext()), mapView);
compassOverlay.enableCompass();
mapView.getOverlays().add(compassOverlay);
```

8. Отображение метрической шкалы масштаба:

```
final Context context = this(getApplicationContext());
final DisplayMetrics dm = context.getResources().getDisplayMetrics();
ScaleBarOverlay scaleBarOverlay = new ScaleBarOverlay(mapView);
scaleBarOverlay.setCentred(true);
scaleBarOverlay.setScaleBarOffset(dm.widthPixels / 2, 10);
mapView.getOverlays().add(scaleBarOverlay)
```

9. Добавление маркера на карту и обработка нажатия:

```
Marker marker = new Marker(mapView);
marker.setPosition(new GeoPoint(55.794229, 37.700772));
marker.setOnMarkerClickListener(new Marker.OnMarkerClickListener() {
    public boolean onMarkerClick(Marker marker, MapView mapView) {
        Toast.makeText(getApplicationContext(), "Click",
        Toast.LENGTH_SHORT).show();
        return true;
    }
});
mapView.getOverlays().add(marker);
marker.setIcon(ResourcesCompat.getDrawable(getResources(), org.osmdroid.li-
brary.R.drawable.osm_ic_follow_me_on, null));
marker.setTitle("Title");
```

Финальным этапом является процесс сборки и запуск приложения. На рисунке 3.3 представлен экран приложения «OSMMaps».

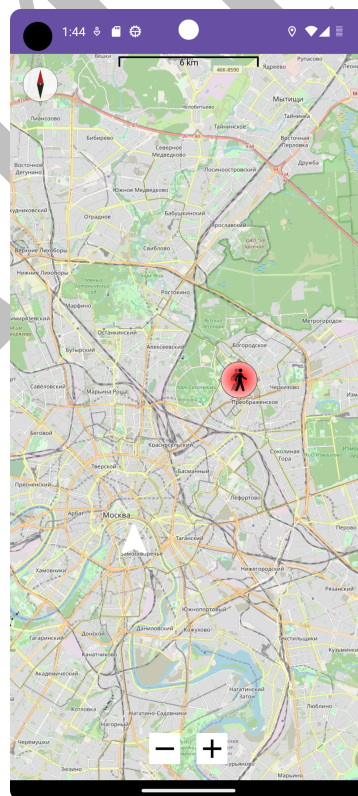


Рисунок 3.3 Экран приложения «OSMMaps»

Для **ВЫПОЛНЕНИЯ ЗАДАНИЯ** требуется добавить все перечисленные выше функции и разместить маркеры в наиболее интересующих местах с описанием.

Мир

4 КОНТРОЛЬНОЕ ЗАДАНИЕ

В контрольном задании «*MireaProject*» требуется создать новый фрагмент «Заведения». Создать экран с различными заведениями, их описанием и возможностью отображения на карте. Возможно использование любого картографического сервиса. При нажатии на маркер отображается адрес и краткое описание. Добавить 1 функцию на выбор по работе с картой.

MIREA